Introduction to JAVA

Concise Study Notes for OAVS Computer Teacher Competitive Exam

Contents

1. History of Java
2. Features of Java
3. JDK Environment
4. The Java Virtual Machine (JVM)
5. Garbage Collection

1. History of Java

Introduction: Java is a general-purpose, class-based, object-oriented programming language originally developed by **James Gosling** and his team at **Sun Microsystems**. It was designed to be simple, portable, and secure, following the philosophy **"Write Once, Run Anywhere" (WORA)**.

Origin and Evolution

- The project began in **June 1991** as part of the **"Green Project"** at Sun Microsystems, aimed at developing software for consumer electronic devices (set-top boxes, digital TVs).
- The language was initially named **"Oak"** after an oak tree outside Gosling's office. It was later renamed **"Green"**, and finally **"Java"** in 1995 — inspired by Java coffee.
- Java was **officially released in May 1995** along with the slogan "Write Once, Run Anywhere."
- Java's syntax is heavily influenced by **C and C++**, but it removed complex features like pointers and multiple inheritance (of classes) to make it simpler and safer.
- In 2009–2010, **Sun Microsystems was acquired by Oracle Corporation**, which continues to develop and maintain Java today.

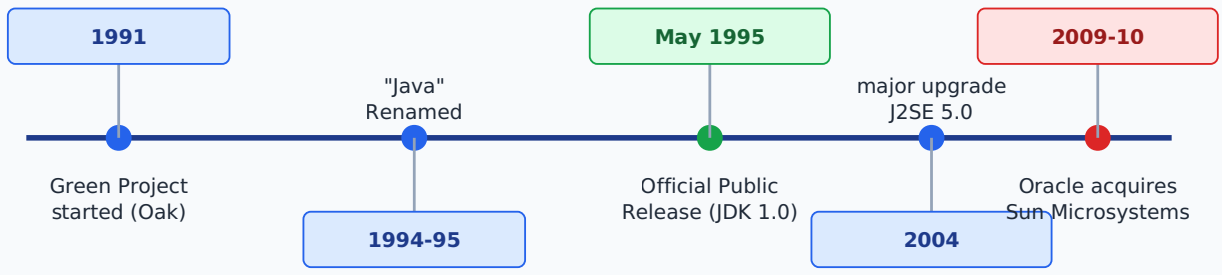


Fig 1.1 — Timeline of Java's evolution from the Green Project to Oracle's ownership

Java Version Naming History

Original Name	Given By	Reason
Oak	James Gosling	Named after an oak tree outside his office
Green	Project team	Internal project code name
Java	Project team	Inspired by Java coffee; because "Oak" was already trademarked by Oak Technology

Key Exam Facts

- Java's creator: **James Gosling** (called the "Father of Java").
- Original name of Java: **Oak**; renamed because the name was already trademarked by **Oak Technology**.
- Green Project started: **1991**; Java released: **1995**.
- Company that developed Java: **Sun Microsystems**; now owned by **Oracle Corporation**.
- Java derives its syntax largely from **C++**.
- Motto/Philosophy: **"Write Once, Run Anywhere" (WORA)**.

Summary

Java originated as the "Green Project" (1991) at Sun Microsystems, was first called Oak, and was renamed Java before its official release in 1995. It was built for portability across devices and is now owned and developed by Oracle Corporation.

2. Features of Java

Introduction: Java's popularity comes from a set of design features (often called "buzzwords") that make it robust, portable, and suitable for distributed and enterprise applications.

Major Features

Feature	Explanation
Simple	Easy syntax similar to C/C++ but without pointers, complex memory management, or operator overloading.
Object-Oriented	Everything (except primitive types) is treated as an object, based on classes, encapsulation, inheritance, and polymorphism.
Platform-Independent	Java source code compiles to **bytecode** (.class files) which can run on any device having a JVM — "Write Once, Run Anywhere."
Secure	No explicit pointers; code runs inside a JVM sandbox; bytecode verifier checks code before execution; supports encryption via APIs.
Robust	Strong memory management, automatic garbage collection, exception handling, and strict compile-time/runtime type checking reduce crashes.
Architecture-Neutral	Bytecode format is not tied to any specific CPU or OS architecture.
Portable	Same bytecode runs on any platform with a compatible JVM; no implementation-dependent data type sizes.
High Performance	Uses **Just-In-Time (JIT) compiler** to convert bytecode into native machine code at runtime for faster execution.
Multithreaded	Supports concurrent execution of two or more parts of a program for maximum utilization of CPU.
Distributed	Provides networking capability (via `java.net`) that supports building distributed applications easily.
Dynamic	Can adapt to an evolving environment; supports dynamic loading of classes at runtime.
Interpreted	Bytecode is interpreted by the JVM, enabling platform independence along with JIT-based speed.

Key Exam Facts

- Java features are commonly called **"Java Buzzwords"** (a term from the official Java White Paper by Sun).
- Platform independence is achieved through **bytecode + JVM**, NOT by direct native compilation.
- Java does **not support pointers** or **multiple inheritance of classes** — this makes it simpler and more secure than C++.
- Total commonly listed buzzwords: **simple, object-oriented, portable, platform-independent, secured, robust, architecture-neutral, multithreaded, interpreted, high performance, distributed, dynamic**.

Summary

> Java combines simplicity, object-orientation, platform independence (via bytecode/JVM), robustness, and security with support for multithreading and networking — making it suitable for both desktop and distributed/enterprise applications.

3. JDK Environment

Introduction: The **JDK (Java Development Kit)** is a complete software development environment used to develop, compile, debug, and run Java applications. It includes development tools, libraries, and a runtime environment.

Components of JDK

- **JRE (Java Runtime Environment)** — provides the environment needed to run Java programs; includes JVM + core class libraries.
- **JVM (Java Virtual Machine)** — part of JRE; executes Java bytecode (detailed in the next topic).
- **Compiler (javac)** — converts `.java` source files into `.class` bytecode files.
- **Loader (java)** — loads the compiled `.class` bytecode file into memory and starts up the JVM to run it.
- **Interpreter** — once loaded, reads and executes the bytecode instructions one by one inside the JVM.
- **Development Tools** — `javadoc` (documentation generator), `jdb` (debugger), `jar` (archiver), `javap` (disassembler), etc.
- **Core Class Libraries (API)** — pre-built packages such as `java.lang`, `java.util`, `java.io`.

JDK vs JRE vs JVM

Aspect	JDK	JRE	JVM
Full Form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Purpose	Develop + run Java programs	Only run Java programs	Executes bytecode
Contains	JRE + compiler + dev tools	JVM + libraries	Class loader + execution engine
Used By	Programmers/ developers	End users	Both (internally)
Platform Dependency	Platform-specific installer	Platform-specific	Platform-specific implementation, abstract specification

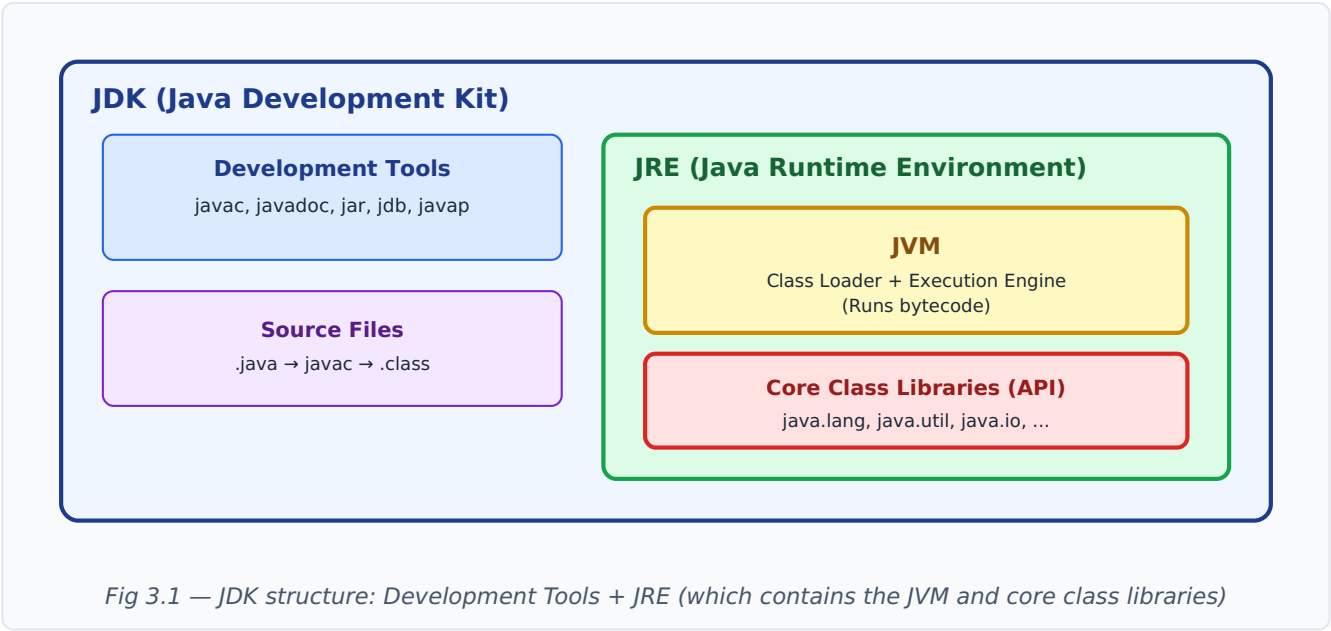


Fig 3.1 — JDK structure: Development Tools + JRE (which contains the JVM and core class libraries)

Compilation and Execution Flow

`.java` source file → compiled by **javac** → produces **.class** bytecode file → executed by **JVM** (inside JRE) → program output.

Key Exam Facts

- Relationship: **JDK ⊃ JRE ⊃ JVM** (JDK is the largest superset).
- To only **run** a Java program, JRE is sufficient; to **develop** one, JDK is required.
- `javac` is the compiler; `java` command launches the JVM to run the class file.
- Java source files must have extension `.java`; compiled files have extension `.class`.

Summary

The JDK is the full development environment containing compiler and tools plus the JRE (runtime + libraries), which in turn contains the JVM that actually executes compiled bytecode.

4. The Java Virtual Machine (JVM)

Introduction: The **JVM** is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. It is the component that gives Java its platform independence, since each OS/platform has its own JVM implementation that understands the same standard bytecode.

Quick meanings: metadata — class/method structure info; **static variables** — shared once per class, not per object; **constant pool** — fixed values like literals and references used by the class; **instance variables** — data unique to each individual object.

Main Components of JVM Architecture

- **Class Loader Subsystem** — loads, links, and initializes `.class` files into memory (Loading → Linking → Initialization).
- **Runtime Data Areas** — memory areas used during execution:
  - **Method Area** — stores class-level data (metadata, static variables, constant pool).
  - **Heap** — stores objects and instance variables; shared across threads; managed by the Garbage Collector.
  - **Stack** — stores frames for method calls (local variables, partial results); one stack per thread.
  - **PC (Program Counter) Register** — holds the address of the currently executing instruction, per thread.
  - **Native Method Stack** — supports native (non-Java) method calls, e.g., C/C++ code.
- **Execution Engine** — executes the bytecode; consists of:
  - **Interpreter** — executes bytecode line by line (slower, repeated interpretation).
  - **JIT (Just-In-Time) Compiler** — compiles frequently used bytecode ("hot code") into native machine code for speed.
  - **Garbage Collector** — reclaims the memory of unused objects from the heap.
- **Native Method Interface (JNI)** — allows Java code to call/be called by native applications and libraries.
- **Native Method Libraries** — platform-specific libraries required by the Execution Engine.

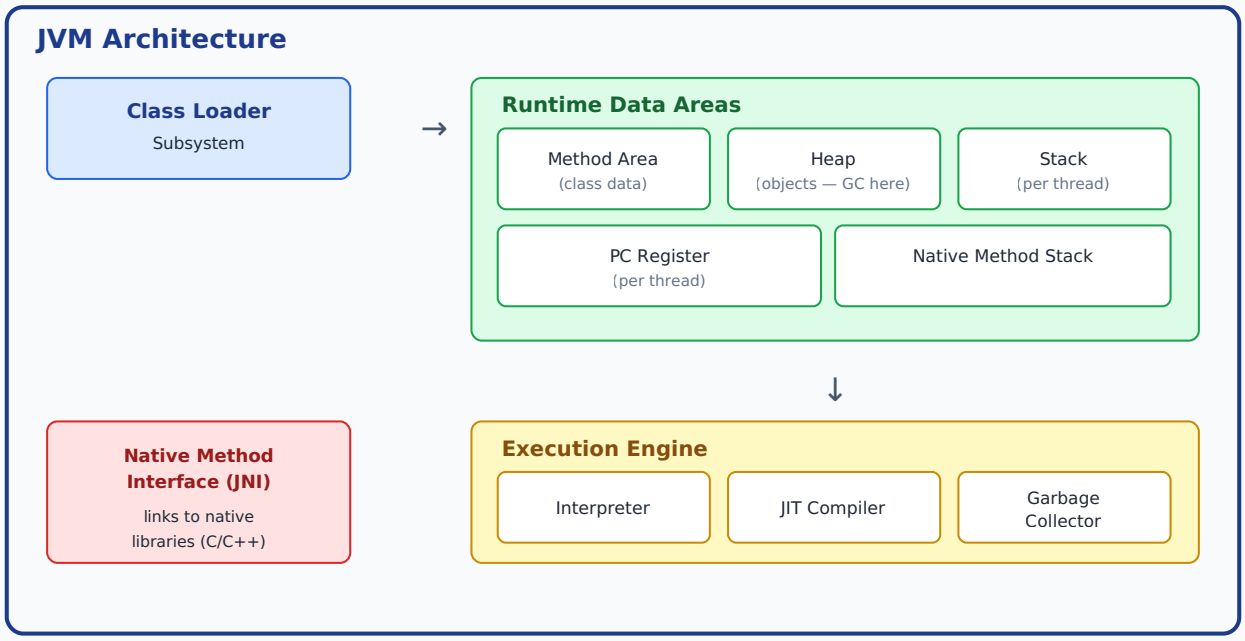


Fig 4.1 — JVM internal architecture: Class Loader → Runtime Data Areas → Execution Engine, with JNI for native code

Execution Flow (Step by Step)

1. `.java` file is compiled by `javac` into `.class` bytecode.
2. **Class Loader** loads the `.class` file into the JVM (Loading → Linking → Initialization).
3. Loaded data is placed into the appropriate **Runtime Data Areas** (Method Area, Heap, Stack, etc.).
4. The **Execution Engine** reads and executes bytecode instructions — using the Interpreter, and the JIT compiler for frequently executed code.
5. The **Garbage Collector** automatically frees heap memory occupied by unreachable objects.
6. Output is produced / program terminates, and the JVM shuts down (or continues for other threads).

Key Exam Facts

- JVM is **platform-dependent in implementation** but the **bytecode it runs is platform-independent** — this is the basis of "Write Once, Run Anywhere."
- JVM is an **abstract/virtual specification**; actual implementations (e.g., HotSpot JVM by Oracle) vary by OS.
- The **Heap** is shared among all threads; the **Stack** and **PC Register** are created per thread.
- JIT compiler improves performance by compiling "hot" bytecode into native code, avoiding repeated interpretation.
- Class loading follows three phases: **Loading, Linking (Verify, Prepare, Resolve), Initialization**.

Summary

The JVM loads bytecode via the Class Loader, stores it in runtime data areas (Method Area, Heap, Stack, PC Register, Native Stack), and executes it using the Execution Engine (Interpreter + JIT + Garbage Collector) — enabling Java's platform independence.

5. Garbage Collection

Introduction: Garbage Collection (GC) is the process by which the JVM automatically identifies and removes objects from the **heap** memory that are no longer reachable/referenced by the program, freeing memory without manual intervention by the programmer (unlike C/C++, where `free()` / `delete` is manual).

Why Garbage Collection is Needed

- Prevents **memory leaks** by automatically reclaiming unused object memory.
- Removes the burden of manual memory management from the programmer, reducing bugs like dangling pointers.
- Improves program **robustness and reliability**.

How Garbage Collection Works

1. **Marking** — GC identifies which objects in the heap are still in use (reachable from "GC roots" like active thread stacks, static references) and which are unreachable (garbage).
2. **Sweeping / Deletion** — unreachable/unmarked objects are removed and their memory is reclaimed.
3. **Compaction** (optional) — remaining objects are moved together to reduce memory fragmentation, making future allocation faster.

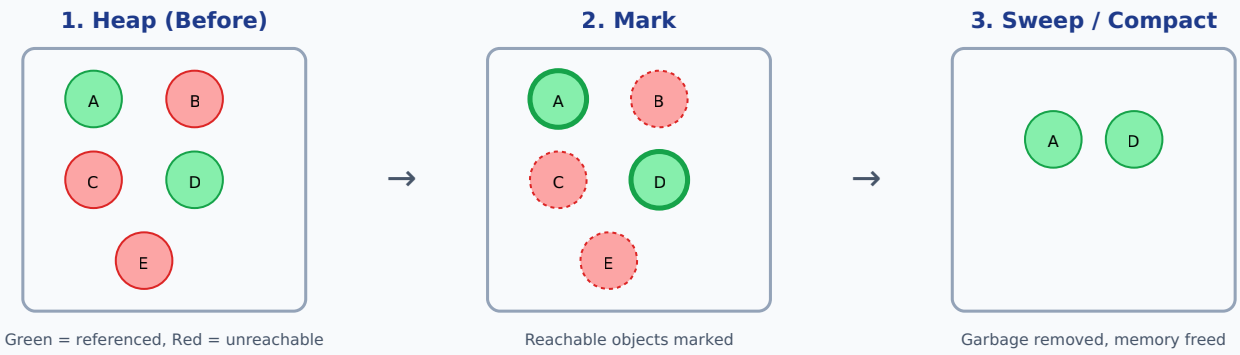


Fig 5.1 — Garbage Collection process: Mark reachable objects → Sweep unreachable ones → (optionally) Compact remaining memory

Key Concepts

Term	Meaning
GC Roots	Starting references (local variables in active stacks, static fields, JNI references) used to determine reachability.
Reachable Object	An object that can be accessed/traced from a GC root — kept in memory.
Unreachable Object	An object with no active references — eligible for garbage collection.
`finalize()`	A method that can be overridden to perform cleanup before an object is collected (deprecated in modern Java, but exam-relevant).
`System.gc()`	Requests (does not guarantee) that the JVM run garbage collection.
Generational GC	Heap is divided into **Young Generation** (new objects), **Old/Tenured Generation** (long-lived objects), and **Permanent Generation/Metaspace** (class metadata) for efficient collection.

Key Exam Facts

- Garbage Collection in Java is **automatic** — a major difference from C/C++, which require manual memory deallocation.
- GC operates on the **Heap** memory area only, not on the Stack.
- Garbage Collector is a part of the JVM's **Execution Engine**.
- An object becomes eligible for GC when it has **no reachable references** (e.g., set to `null`, goes out of scope).
- `System.gc()` only **suggests** garbage collection to the JVM; it does not force it immediately.
- Common GC algorithm approach tested in exams: **Mark and Sweep**.

Summary

Garbage Collection automatically frees heap memory occupied by objects that are no longer referenced, using a Mark-and-Sweep (with optional Compaction) approach, thereby improving memory management and program robustness without manual intervention.

Quick Revision — Key Facts for Exam Day

1. History of Java

- Creator: James Gosling | Company: Sun Microsystems | Now owned by: Oracle
- Original name: Oak (1991, Green Project) → renamed Java → released 1995
- Motto: "Write Once, Run Anywhere"

2. Features of Java

- Simple, Object-Oriented, Platform-Independent, Secure, Robust, Architecture-Neutral, Portable, High-Performance, Multithreaded, Distributed, Dynamic, Interpreted
- Platform independence = Bytecode + JVM

3. JDK Environment

- JDK = JRE + Development Tools (javac, javadoc, jar, jdb)
- JRE = JVM + Class Libraries
- Flow: .java → javac → .class → JVM executes

4. JVM

- Parts: Class Loader → Runtime Data Areas (Method Area, Heap, Stack, PC Register, Native Stack) → Execution Engine (Interpreter, JIT, GC)
- JVM = platform-dependent implementation; bytecode = platform-independent
- Class loading phases: Loading → Linking → Initialization

5. Garbage Collection

- Automatic memory management on the Heap by the JVM
- Process: Mark → Sweep → (Compact)
- `System.gc()` only requests GC, doesn't force it
- Generations: Young, Old/Tenured, Metaspace

Quick Fact	Answer
Java released in	1995
Java's original name	Oak
Java syntax based on	C++
Compiler command	javac
Interpreter/launcher command	java
Component that runs bytecode	JVM
Superset relationship	JDK ⊃ JRE ⊃ JVM
Memory area GC works on	Heap
GC algorithm (basic)	Mark and Sweep